Screenshots — Terraform Advanced Commands

Lecture 3 · 58 shots, chronological · source stills from the session (2026-09-24 15:06 – 19:28)

Terraform Advance Command

With the basics covered, let's move on to some slightly more advanced CLI topics.

- terraform graph # **This** is used to generate a **visual representation of the resources** in your infrastructure and how they are **connected or depend** on each other.

- terraform taint # This is used to manually mark a specific resource as **tainted**. When a resource is tainted, it means Terraform will **consider it damaged or needing replacement** on the next apply operation.
- terraform import # This command used to **bring existing infrastructure** (that wasn't originally created with Terraform) under Terraform's management.
- terraform workspace # Terraform workspaces enable us to **manage multiple deployments** of the same configuration.
- terraform refresh # terraform refresh **reads the current settings** from all managed remote objects and updates the Terraform state to match.
- terraform logging # Terraform logging **records the internal steps** Terraform takes when managing your infrastructure.
- terraform provisioners # Terraform Provisioners are tools used within a Terraform configuration to **execute scripts or** commands on a local or remote machine as part of the resource provisioning process.

- terraform plugin cache # Terraform plugin cache is like a **local storage that saves plugins** to make Terraform faster and more reliable.

Ram Misra

Command Comparison

Capability	Legacy Approach (`terraform taint`)	Modern Approach (Recommended)
Command Syntax	`terraform taint <RESOURCE_ADDRESS>`	`terraform apply -replace="<RESOURCE_ADDRESS>"`
State File Impact	Modifies the state file immediately.	Only modifies the state during execution.
Execution Risk	High (creates a hidden "ticking time bomb" in shared state).	Low (safe for simultaneous collaborative environments).
Status	Deprecated since v0.15.2.	Fully supported in stable versions.

```
root@cka-w2:~/project1# terraform state list
aws_instance.web
aws_s3_bucket.my_s3_bucket
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# terraform taint aws_instance.web
Resource instance aws_instance.web has been marked as tainted.
root@cka-w2:~/project1#
```

```
{
  "version": 4,
  "terraform_version": "1.16.3",
  "serial": 4,
  "lineage": "7a41593c-e49e-381c-29d4-18bd36dea709",
  "outputs": {},
  "resources": [
    {
      "mode": "managed",
      "type": "aws_instance",
      "name": "web",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "status": "tainted",
          "schema_version": 2,
          "attributes": {
            "ami": "ami-003c4e453d6057f7d",
            "arn": "arn:aws:ec2:ap-south-1:749771281136:instance/i-05ef7355daff
edde3",
```

```
root@cka-w2:~/project1# terraform untaint aws_instance.web
Resource instance aws_instance.web has been successfully untainted.
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# terraform taint aws_instance.web
Resource instance aws_instance.web has been marked as tainted.
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# terraform apply --auto-approve
```

```
1 to add, 0 to change, 1 to destroy.
```

What is terraform import?

terraform import is a command in Terraform used to bring existing infrastructure (that wasn't originally created with Terraform) under Terraform's management. It maps real-world infrastructure resources into your Terraform state file so Terraform can manage them going forward.

⚠ **Important**: It **does not** automatically generate the corresponding .tf configuration files; you must write those manually or use tools to help generate them.

Syntax

terraform import [options] ADDRESS ID·**ADDRESS** – Terraform resource address (e.g., aws_instance.example)

·**ID** – ID of the resource in the provider's platform (e.g., EC2 instance ID)

Instance summary for i-0c27f664619a950cc (Terraform-import-ec2) Info

Connect | Instance state | Actions

Updated less than a minute ago

Instance ID
i-0c27f664619a950cc

IPv6 address
–

Hostname type
IP name: ip-172-31-3-56.ap-south-1.compute.internal

Answer private resource DNS name
IPv4 (A)

Auto-assigned IP address
13.203.214.50 [Public IP]

IAM role
–

IMDSv2
Required

Operator
–

Public IPv4 address
13.203.214.50 | open address

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-3-56.ap-south-1.compute.internal

Instance type
t2.nano

VPC ID
vpc-0f3a65eee58b3e83f

Subnet ID
subnet-01d00bb0f998e78aa

Instance ARN
arn:aws:ec2:ap-south-1:749771281136:instance/i-0c27f664619a950cc

Private IPv4 addresses
172.31.3.56

Public DNS
ec2-13-203-214-50.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses
–

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name
–

Managed
false

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Instance details Info

AMI ID
ami-066c4849e6b3a1e3d

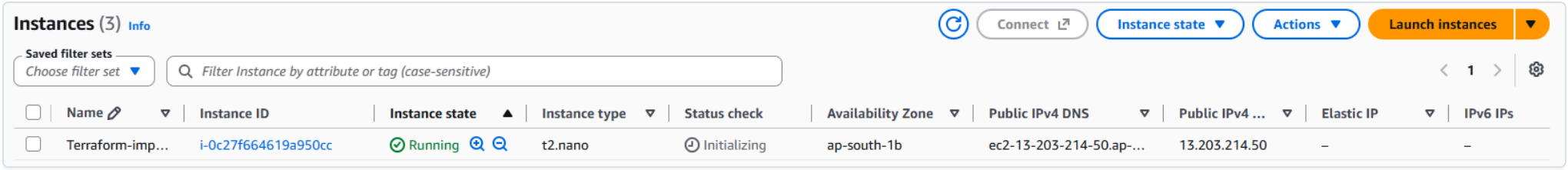
AMI name
al2023-ami-2023.12.20260918.0-kernel-6.18-x86_64

Monitoring
disabled

Allowed image
-

Platform details
Linux/UNIX

Termination protection
Disabled


Instances (3) Info
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
1
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Public IPv4 ...
Elastic IP
IPv6 IPs
Terraform-imp...
i-0c27f664619a950cc
Running
t2.nano
Initializing
ap-south-1b
ec2-13-203-214-50.ap-...
13.203.214.50
–
–

```
root@cka-w2:~/project1# ls
ec2-tf
root@cka-w2:~/project1# cat ec2-tf
resource "aws_instance" "terraform-import" {
}

root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# ls
ec2-tf
root@cka-w2:~/project1# cat ec2-tf
resource "aws_instance" "terraform-import" {
}

root@cka-w2:~/project1# terraform import aws_instance.terraform-import i-0c27f6
64619a950cc
```

```
root@cka-w2:~/project1# terraform import aws_instance.terraform-import i-0c27f664619a950cc

aws_instance.terraform-import: Import prepared!
  Prepared aws_instance for import


Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.

root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# ls -l
total 12
-rw-r--r-- 1 root root   48 Sep 24 10:24 ec2.tf
-rw-r--r-- 1 root root 5477 Sep 24 10:30 terraform.tfstate
root@cka-w2:~/project1# terraform state list
aws_instance.terraform-import
root@cka-w2:~/project1# terraform show
# aws_instance.terraform-import:
resource "aws_instance" "terraform-import" {
    ami                                  = "ami-066c4849e6b3a1e3d"
    arn                                  = "arn:aws:ec2:ap-south-1:749771281136
:instance/i-0c27f664619a950cc"
    associate_public_ip_address          = true
    availability_zone                    = "ap-south-1b"
    disable_api_stop                     = false
    disable_api_termination              = false
    ebs_optimized                        = false
    force_destroy                        = false
    get password data                    = false
```

```
root@cka-w2:~/project1# cat ec2.tf
resource "aws_instance" "terraform-import" {
}

root@cka-w2:~/project1# terraform apply --auto-approve
```

```
Error:

  with aws_instance.terraform-import,
  on ec2.tf line 1, in resource "aws_instance" "terraform-import":
   1: resource "aws_instance" "terraform-import" {

"launch_template": one of `ami,instance_type,launch_template` must be
specified


Error:

  with aws_instance.terraform-import,
  on ec2.tf line 1, in resource "aws_instance" "terraform-import":
   1: resource "aws_instance" "terraform-import" {

"ami": one of `ami,launch_template` must be specified
```

4. **Reconcile Drift**: Run `terraform plan` . If your `.tf` file code lacks properties that exist in reality, the terminal output will alert you to the discrepancies. Fill them into your configuration block until `terraform plan` reports **"No changes. Infrastructure is up-to-date."**. YouTube · Rahul Wagh +4

hcl

```hcl
import {
  to = aws_instance.web_server
  id = "i-0123456789abcdef0"
}
```

```
resource "aws_instance" "terraform-import" {

ami                                    = "ami-066c4849e6b3a1e3d"
instance_type                          = "t2.nano"
key_name                               = "Terraform-import"

}

~
~
~
~
```

```
root@cka-w2:~/project1# terraform fmt ec2.tf
ec2.tf
root@cka-w2:~/project1# cat ec2.tf
resource "aws_instance" "terraform-import" {

  ami           = "ami-066c4849e6b3a1e3d"
  instance_type = "t2.nano"
  key_name      = "Terraform-import"

}
```

```
  ~ resource "aws_instance" "terraform-import" {
        id                                   = "i-0c27f664619a950cc"
      ~ tags                                 = {
          - "Name" = "Terraform-import-ec2" -> null
        }
      ~ tags_all                             = {
          - "Name" = "Terraform-import-ec2" -> null
        }
      + user_data_replace_on_change          = false
        # (38 unchanged attributes hidden)

        # (9 unchanged blocks hidden)
    }

      0 to add, 1 to change, 0 to destroy.
```

```
Note: You didn't use the -out option to save this plan, so Terraform can't
guarantee to take exactly these actions if you run "terraform apply" now.
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# cat ec2.tf
resource "aws_instance" "terraform-import" {

  ami             = "ami-066c4849e6b3a1e3d"
  instance_type = "t2.nano"
  key_name        = "Terraform-import"

tags = {
    Name = "Terraform-import-ec2"
  }

}

root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# terraform plan
```

```
root@cka-w2:~/project1# terraform plan


No changes.

Terraform has compared your real infrastructure against your configuration and
found no differences, so no changes are needed.
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# vim ec2.tf
root@cka-w2:~/project1# cat ec2.tf
resource "aws_instance" "terraform-import" {

  ami           = "ami-066c4849e6b3a1e3d"
  instance_type = "t2.micro"
  key_name      = "Terraform-import"

  tags = {
    Name = "Terraform-import-ec2"
  }

}

root@cka-w2:~/project1#
```

Instance summary for i-0c27f664619a950cc (Terraform-import-ec2) Info

Updated less than a minute ago

Connect | Instance state | Actions

Instance ID
i-0c27f664619a950cc

IPv6 address
–

Hostname type
IP name: ip-172-31-3-56.ap-south-1.compute.internal

Answer private resource DNS name
IPv4 (A)

Auto-assigned IP address
13.203.214.50 [Public IP]

IAM role
–

IMDSv2
Required

Operator
–

Public IPv4 address
13.203.214.50 | open address

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-3-56.ap-south-1.compute.internal

Instance type
t2.nano

VPC ID
vpc-0f3a65eee58b3e83f

Subnet ID
subnet-01d00bb0f998e78aa

Instance ARN
arn:aws:ec2:ap-south-1:749771281136:instance/i-0c27f664619a950cc

Private IPv4 addresses
172.31.3.56

Public DNS
ec2-13-203-214-50.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses
–

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name
–

Managed
false

```
root@cka-w2:~/project1# cat ec2.tf
resource "aws_instance" "terraform-import" {

  ami           = "ami-066c4849e6b3a1e3d"
  instance_type = "t2.micro"
  key_name      = "Terraform-import"

  tags = {
    Name = "Terraform-import-ec2"
  }

}

root@cka-w2:~/project1# terraform apply -auto-approve
```

```
                                    will be updated in-place
 ~ resource "aws_instance" "terraform-import" {
       id                                    = "i-0c27f664619a950cc"
     ~ instance_type                         = "t2.nano" -> "t2.micro"
     ~ public_dns                            = "ec2-13-203-214-50.ap-south-1.co
pute.amazonaws.com" -> (known after apply)
     ~ public_ip                             = "13.203.214.50" -> (known after
pply)
       tags                                  = {
           "Name" = "Terraform-import-ec2"
       }
     + user_data_replace_on_change           = false
       # (36 unchanged attributes hidden)

       # (9 unchanged blocks hidden)
   }

     0 to add, 1 to change, 0 to destroy.
```

```
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
root@cka-w2:~/project1#
```

Console after — t2.micro, same instance ID, new public IP 13.235.75.134

Instance summary for i-0c27f664619a950cc (Terraform-import-ec2) Info

Updated less than a minute ago

Connect | Instance state | Actions

Instance ID
i-0c27f664619a950cc

IPv6 address
–

Hostname type
IP name: ip-172-31-3-56.ap-south-1.compute.internal

Answer private resource DNS name
IPv4 (A)

Auto-assigned IP address
13.235.75.134 [Public IP]

IAM role
–

IMDSv2
Required

Operator
–

Public IPv4 address
13.235.75.134 | open address

Instance state
Running

Private IP DNS name (IPv4 only)
ip-172-31-3-56.ap-south-1.compute.internal

Instance type
t2.micro

VPC ID
vpc-0f3a65eee58b3e83f

Subnet ID
subnet-01d00bb0f998e78aa

Instance ARN
arn:aws:ec2:ap-south-1:749771281136:instance/i-0c27f664619a950cc

Private IPv4 addresses
172.31.3.56

Public DNS
ec2-13-235-75-134.ap-south-1.compute.amazonaws.com | open address

Elastic IP addresses
–

AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name
–

Managed
false

Terraform under Terraform's management.

- terraform workspace # Terraform workspaces enable us to **manage multiple deployments** of the same configuration.

Terraform workspaces allow you to manage separate, isolated state files for different environments (like `dev`, `staging`, or `prod`) using the exact same configuration files. YouTube · Abhishek.Veeramalla +1

```
root@cka-w2:~/project1# terraform workspace
Usage: terraform [global options] workspace

  new, list, show, select and delete Terraform workspaces.

Subcommands:
    delete    Delete a workspace
    list      List Workspaces
    new       Create a new workspace
    select    Select a workspace
    show      Show the name of the current workspace
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# terraform workspace new Dev-team
Created and switched to workspace "Dev-team"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# terraform workspace new Dev-team
Created and switched to workspace "Dev-team"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
root@cka-w2:~/project1# terraform workspace new UAT-team
Created and switched to workspace "UAT-team"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
root@cka-w2:~/project1# terraform workspace new Prod-team
Created and switched to workspace "Prod-team"!

You're now on a new, empty workspace. Workspaces isolate their state,
so if you run "terraform plan" Terraform will not see any existing state
for this configuration.
```

```
root@cka-w2:~/project1# terraform workspace list
  default
  Dev-team
* Prod-team
  UAT-team
```

```
root@cka-w2:~/project1# terraform workspace list
  default
  Dev-team
* Prod-team
  UAT-team

root@cka-w2:~/project1# terraform workspace show
Prod-team
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# ls -ltrha
total 20K
-rw-r--r-- 1 root root 1.5K Sep 24 09:49 .terraform.lock.hcl
drwx------ 6 root root 4.0K Sep 24 10:53 ..
drwxr-xr-x 3 root root 4.0K Sep 24 12:45 .terraform
drwxr-xr-x 4 root root 4.0K Sep 24 12:45 .
drwxr-xr-x 5 root root 4.0K Sep 24 12:46 terraform.tfstate.d
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# terraform workspace show
Prod-team
root@cka-w2:~/project1# terraform workspace list
  default
  Dev-team
* Prod-team
  UAT-team

root@cka-w2:~/project1# terraform workspace select Dev-team
Switched to workspace "Dev-team".
root@cka-w2:~/project1# terraform workspace show
Dev-team
root@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# cat main.tf
resource "aws_s3_bucket" "example" {
  bucket = "my-tf-test-bucket-0788-${terraform.workspace}"

  tags = {
    Name        = "My bucket-${terraform.workspace}"
  }
}
```

```
      + server_side_encryption_configuration (known after apply)

      + versioning (known after apply)

      + website (known after apply)
    }

      1 to add, 0 to change, 0 to destroy.



Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

```
root@cka-w2:~/project1# tree terraform.tfstate.d/
terraform.tfstate.d/
├── dev-team
│   └── terraform.tfstate
├── Dev-team
│   └── terraform.tfstate
├── Prod-team
└── UAT-team

5 directories, 2 files
root@cka-w2:~/project1#
```

- terraform refresh # terraform refresh **reads the current settings** from all managed remote objects and updates the Terraform state to match.


[root@Devops-server project1]# terraform refresh
aws_instance.linux: Refreshing state... [id=i-048ef6b5b8412181a]
[root@Devops-server project1]#

- terraform logging # Terraform logging **records the internal steps** Terraform takes when managing your infrastructure.


Set the TF_LOG variable

You can set `TF_LOG` to one of the log levels (in order of decreasing verbosity) `TRACE`, `DEBUG`, `INFO`, `WARN` or `ERROR` to change the verbosity of the logs.

Logging Levels in Terraform

Terraform supports several levels of logging, controlled using the TF_LOG environment variable:

You can set the TF_LOG variable to one of the following verbosity levels, from least to most verbose:

- **OFF**: Turns off logging.
- **ERROR**: Only the least verbose output, typically user-facing errors.
- **WARN**: Includes warnings about deprecations or potential issues.
- **INFO**: High-level messages about the execution process.
- **DEBUG**: More detailed information, useful for general debugging.
- **TRACE**: The most verbose level, providing low-level execution steps and internal logs (recommended when reporting bugs

```
export TF_LOG="TRACE"
export TF_LOG_PATH="./terraform.log"
```

```
oot@cka-w2:~/project1# vim /root/.bashrc
oot@cka-w2:~/project1# source /root/.bashrc
oot@cka-w2:~/project1#
```

```
root@cka-w2:~/project1# ls -ltrha
total 20K
-rw-r--r-- 1 root root 1.5K Sep 24 09:49 .terraform.lock.hcl
drwxr-xr-x 3 root root 4.0K Sep 24 12:45 .terraform
drwx------ 6 root root 4.0K Sep 24 13:48 ..
drwxr-xr-x 3 root root 4.0K Sep 24 13:49 .
-rw-r--r-- 1 root root 2.9K Sep 24 13:49 terraform.log
root@cka-w2:~/project1#
```

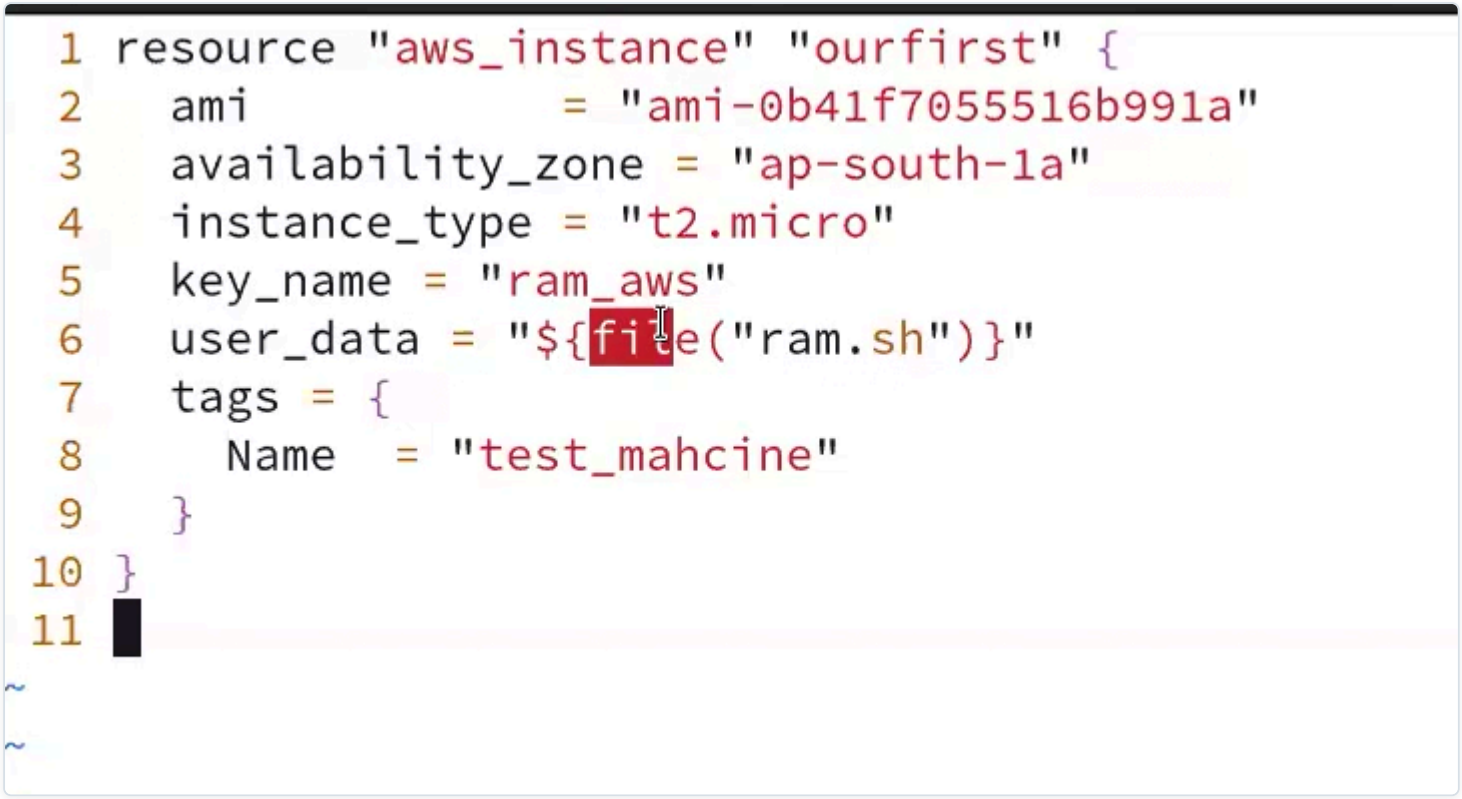
```
root@cka-w2:~/project1# less terraform.log
2026-09-24T13:49:54.444Z [INFO]  Terraform version: 1.16.3
2026-09-24T13:49:54.444Z [DEBUG] using github.com/hashicorp/go-tfe v1.110.0
2026-09-24T13:49:54.444Z [DEBUG] using github.com/hashicorp/hcl/v2 v2.24.0
2026-09-24T13:49:54.444Z [DEBUG] using github.com/hashicorp/terraform-svchost v
0.2.1
2026-09-24T13:49:54.444Z [DEBUG] using github.com/zclconf/go-cty v1.18.1
2026-09-24T13:49:54.444Z [INFO]  Go runtime version: go1.26.8
2026-09-24T13:49:54.444Z [INFO]  CLI args: []string{"terraform", "plan"}
2026-09-24T13:49:54.444Z [TRACE] Stdout is a terminal of width 79
2026-09-24T13:49:54.444Z [TRACE] Stderr is a terminal of width 79
2026-09-24T13:49:54.444Z [TRACE] Stdin is a terminal
```

- terraform user data # Terraform user data is **kind of instructions** you give a new server to run automatically at startup.

```
[root@Devops-server home]# cat ram.sh
#!/bin/bash
sudo yum install httpd -y
        sudo systemctl start httpd
        sudo systemctl enable httpd
        echo "sample webserver using terraform" > /var/www/html/index.html

sudo hostnamectl set-hostname web_server
```

```
[root@Devops-server project1]# ll
total 20
-rw-r--r--. 1 root root  253 Jun 22 09:18 main.tf
-rw-r--r--. 1 root root  227 Jun 22 09:27 ram.sh
-rw-r--r--. 1 root root  181 Jun 22 09:19 terraform.tfstate
-rw-r--r--. 1 root root 5493 Jun 22 09:19 terraform.tfstate.backup
[root@Devops-server project1]#
```


1 resource "aws_instance" "ourfirst" {
2 ami = "ami-0b41f7055516b991a"
3 availability_zone = "ap-south-1a"
4 instance_type = "t2.micro"
5 key_name = "ram_aws"
6 user_data = "${file("ram.sh")}"
7 tags = {
8 Name = "test_mahcine"
9 }
10 }
11